

Multi-Agent Debate System

Two AI agents debate a topic. A neutral Judge decides when the debate is over and delivers a clear decision with confidence score.

1. Simple Explanation (For Everyone)

Imagine you have an important decision to make, but you only hear one side of the story.

Now imagine you invite **two smart people** who disagree with each other to argue the topic in front of you. One strongly supports the idea. The other strongly opposes it. They go back and forth for several rounds.

After some time, a **neutral referee** steps in, looks at everything that was said, and tells you:

- Which side made the stronger case
- How confident they are in that conclusion
- A clear summary table of the key points

That is exactly what this system does — but with AI agents instead of humans.

Why this is useful

- Reduces one-sided or biased answers from a single AI
- Forces deeper reasoning through disagreement
- Produces a structured, decision-ready output instead of a long wall of text
- Can stop early when the answer is already clear

2. What the System Does

Component	Role
Agent Pro	Argues <i>in favor</i> of the topic
Agent Con	Argues <i>against</i> the topic

Judge	Evaluates the debate every 2 rounds and decides when to stop
--------------	--

Key Features

- **Clarity Check** first — if the topic is vague, it asks clarifying questions
- **Dynamic temperature** — agents are more creative early and more focused later
- **Early stopping** — debate can end before the maximum rounds, but only once the Judge is both past `min_rounds` *and* reports confidence at or above the configured threshold
- **Minimum rounds** — prevents stopping too early
- **Full transcript** saved as a clean Markdown file (including partial transcripts if a run fails midway)
- **Confidence score** (0–100%) with reasoning
- **Summary table** of strongest points and weaknesses
- **Opponent-aware agents** — each side sees the other's arguments as an opponent's statements, not as its own prior output, so rebuttals engage with what was actually said

3. When to Use It

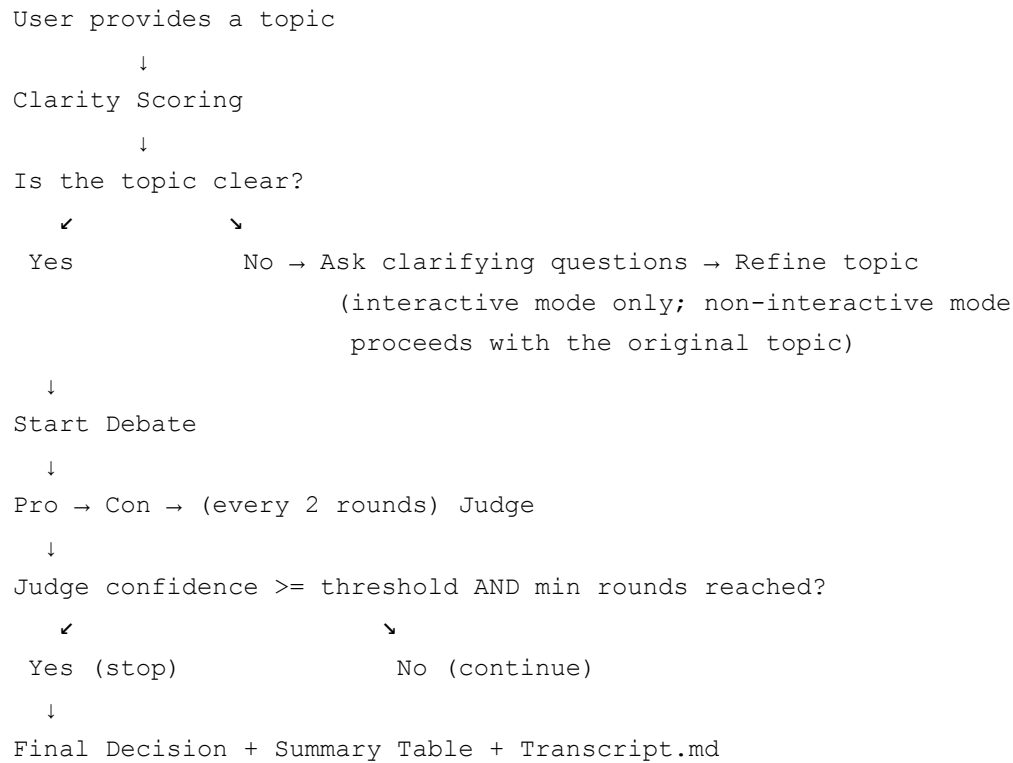
Good use cases:

- Important product or architecture decisions
- Evaluating controversial or high-stakes topics
- Reducing bias when asking an AI for advice
- Generating balanced analysis for reports or presentations
- Exploring both sides of a business, ethical, or technical question

Not ideal for:

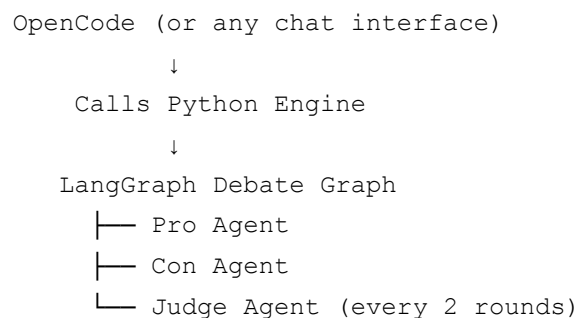
- Simple factual questions
 - Creative writing or brainstorming (use other tools)
 - Extremely time-sensitive answers
-

4. How the Workflow Works



5. Technical Architecture

Recommended Hybrid Design



Why hybrid?

- OpenCode / UI gives a nice experience
- Python + LangGraph gives precise control, early stopping, structured output, and reproducibility

For a UI/API caller specifically, use the two-phase clarification functions (`get_clarifying_questions` then `resolve_topic`) rather than `run_debate`, since `run_debate`'s interactive mode blocks on `input()` and isn't suitable behind a request/response boundary. Pass `non_interactive=True` to `run_debate` (or use `run_debate_graph` directly) if you don't want to implement the clarification round-trip.

Core Technologies

- **LangGraph** — stateful multi-agent orchestration
 - **Pydantic** — structured outputs (Judge decision, Clarity assessment)
 - **LiteLLM / OpenAI-compatible API** — model flexibility via `--model` / `--base-url` / `--api-key` CLI flags (or `DEBATE_MODEL` / `OPENAI_BASE_URL` / `OPENAI_API_KEY` env vars)
 - Dynamic temperature scaling
 - Markdown transcript generation
-

6. Implementation Details

Clarity Scoring Logic

Before the debate starts, the system scores the topic from 0.0 to 1.0.

- ≥ 0.72 → Start debate immediately
- < 0.72 → Ask 1–3 clarifying questions, then rewrite a clearer version of the topic (interactive mode), or proceed with the original topic (non-interactive mode)

Dynamic Temperature

```
Early rounds → higher temperature (more exploratory)
Later rounds → lower temperature (more convergent)
```

Formula used:

```
temp = 0.85 - (progress * 0.55)  # clamped between 0.30 and 0.85
```

Judge Behavior

- Runs only after even rounds (2, 4, 6...)

- Can only stop the debate once **both** `min_rounds` is reached **and** reported confidence is at or above `EARLY_STOP_CONFIDENCE` (default 0.82) — both conditions are enforced in code, not just requested via prompt
- Returns:
 - Winner (Pro / Con / Draw)
 - Confidence score
 - Reasoning
 - Summary Markdown table

Message Roles

Each agent's view of the transcript is built relative to itself: its own prior turns are treated as assistant messages, and everything else (opponent turns and Judge remarks) is treated as user messages. This keeps Pro and Con arguing with each other instead of each side effectively continuing its own monologue.

Error Handling

LLM calls are retried up to 3 times with backoff. If a run still fails (e.g. persistent API outage), the transcript gathered so far is saved to `debate_transcripts/..._PARTIAL.md` before the error is raised, so no in-progress debate is silently lost.

Output

- Live progress in the terminal
- Final decision + confidence
- Summary table
- Full transcript saved to `debate_transcripts/debate_YYYYMMDD_HHMMSS_topic.md` (or `..._PARTIAL.md` on failure)

7. How to Run

```
# Basic usage
python debate_engine.py --topic "Should companies fully replace junior developers"

# Force longer debate
python debate_engine.py --topic "Is remote work better than office work?" --min-r
```

```
# Cap total rounds too
python debate_engine.py --topic "..." --min-rounds 4 --max-rounds 8

# Point at a LiteLLM proxy or other OpenAI-compatible endpoint
python debate_engine.py --topic "..." --model gpt-4o --base-url http://localhost:

# Skip interactive clarifying-question prompts (for scripted/CI use)
python debate_engine.py --topic "..." --non-interactive
```

Requirements

```
pip install -r requirements.txt
```

(pins langgraph , langchain-openai , langchain-core , pydantic)

8. Design Decisions Summary

Decision	Reason
Two opposing agents	Forces deeper reasoning through conflict
Separate Judge	Neutral evaluation + structured decision
Judge every 2 rounds	Balance between cost and control
min_rounds	Prevents premature conclusions
Confidence threshold	Prevents stopping on a shaky/unclear read, enforced in code
Clarity check first	Improves debate quality on vague topics
Dynamic temperature	Exploration early, precision later
Per-agent message roles	Keeps Pro/Con genuinely responding to each other
Retry + partial save	Survives transient API failures without losing progress
Markdown transcript	Easy to share, review, and archive

9. Future Improvements

- Stream the debate live into a web UI (the `on_update` callback in `run_debate_graph` is a starting point)
 - Support different persona pairs (Technical vs Business, Optimistic vs Pessimistic)
 - Voting mechanism between multiple judges
 - Cost & token tracking
 - Web UI with real-time agent messages
-

Document generated for the Multi-Agent Debate System project.